PROJECT 1

3D AI Visualization

Detailed Free-Software Architecture, Installation, Development, Testing and Deployment Guide

Purpose: A separate visualization application that can later observe an AI system. It is NOT part of Project 2 — Adaptive Personal AI. It does not contain the AI brain, memory, RAG or agent reasoning.

Version 1.0 • September 2026

1. Project Definition

Project 1 is a lightweight, stylized visualization client. Its job is to make system state understandable visually: active roles, tasks, communication, progress, memory/learning events and status. It is not a realistic city, not a simulation of hundreds of NPCs, and not a requirement for the Personal AI to operate.

The visual language should be simple/low-detail/isometric/grid-like: simple geometry, simple characters/icons, minimal textures and lighting, clear labels and status indicators. The visual client can be opened or closed independently.

2. Fixed Scope

- Lightweight 3D scene and UI.
- Visualize AI roles as simple characters, icons or stations.
- Visualize active/inactive/busy/error states.
- Visualize task flow and communication.
- Visualize selected memory/learning events.
- Connect to a backend/event source later.
- Run entirely without the AI backend using mock data.
- Export as a Windows application and optionally a Web build.

3. Non-Goals

- Photorealistic graphics.
- Large autonomous city simulation.
- Hundreds of AI citizens.
- AI reasoning inside Godot.
- Storing the authoritative AI memory in the visualization.
- Making the visualization necessary for Project 2.
- Paid game engines, paid asset subscriptions or paid deployment services.

4. Recommended Free Stack

Technology	Selection	Why
Engine	Godot 4.7.2 stable	Free/open-source, strong 2D/3D support, lightweight and suitable for stylized visualization.
3D asset creation	Blender 5.2 LTS	Free/open-source; can create simple low-poly assets and export standard formats.
Language	GDScript	Built into Godot; fast to learn and avoids another runtime.
Version control	Git + GitHub Free	Source history, releases and collaboration.
Web deployment	GitHub Pages	Free static hosting for public repositories; suitable for a web export.
Desktop distribution	GitHub Releases / direct ZIP	No hosting server required for a Windows build.
Optional editor	VS Code	Free source editor; useful for scripts and JSON/API work.

Godot is MIT-licensed; Blender is GPL/free software and the Blender Foundation states that artwork you create is yours to use. Current Godot stable is 4.7.2 as of August 2026; use a stable release rather than a development build.

5. Why Godot

- Free and open source, avoiding engine subscription cost.
- Supports Windows and Web export.
- Good scene/node system for visual state machines.

- GDScript is concise for UI, animation and event visualization.
- Low-level graphics complexity is hidden behind a practical editor.
- Easy to keep the visualization lightweight enough for the target laptop.

6. Why Blender

- Free/open-source modeling tool.
- Excellent for simple low-poly characters, buildings, icons and props.
- Can be used offline.
- Exports assets that Godot can consume.
- Allows you to create your own reusable asset library without a paid asset pipeline.

7. Installation on Windows

Install only the free tools required for the project.

1. Install Git for Windows.
2. Install Godot 4.7.2 stable.
3. Install Blender 5.2 LTS if custom 3D assets are required.
4. Install VS Code if you want a general-purpose code editor.
5. Create a dedicated project directory such as C:\AIProjects\project1-3d-visualization.

Verify Git:

```
git --version
```

Verify Godot from a terminal if the executable is on PATH:

```
godot --version
```

Verify Blender if configured on PATH:

```
blender --version
```

If an executable is not on PATH, launch it from its installation folder; PATH configuration is optional for GUI development.

8. Project Folder Structure

```
project1-3d-ai-visualization/
■■■ godot/
■ ■■■ project.godot
■ ■■■ scenes/
■ ■■■ scripts/
■ ■ ■■■ state/
■ ■ ■■■ agents/
■ ■ ■■■ tasks/
■ ■ ■■■ ui/
■ ■ ■■■ networking/
■ ■■■ assets/
■ ■ ■■■ characters/
■ ■ ■■■ buildings/
■ ■ ■■■ icons/
■ ■ ■■■ materials/
■ ■■■ mock/
■ ■■■ tests/
■■■ blender/
■ ■■■ source/
■ ■■■ exports/
■■■ docs/
■■■ builds/
■■■ README.md
```

9. Core Architecture

```
PROJECT 1
|
+---------+---------+
| |
Visual State UI State
| |
+---------+---------+
|
Godot Scene Tree
|
+----------+----------+
| | |
Agents Tasks Events
| | |
+----------+----------+
|
Optional Network/API
|
External AI Backend

Authoritative AI logic stays OUTSIDE this project.
```

10. Scene Design

- Main scene: root visualization world.
- Agent nodes: one reusable scene for each displayed role.
- Task nodes: reusable visual task cards/markers.
- Event stream: UI list/timeline for recent events.
- Status panel: selected agent/task details.
- Camera controller: fixed isometric or controlled free camera.
- Environment: simple grid/platform rather than a detailed city.

11. State Model

Field	Example
agent_id	research_01
role	Research Agent
state	idle / working / waiting / error / sleeping
task_id	task_102
progress	0.0–1.0
current_action	Collecting sources
message	Waiting for evidence
timestamp	ISO timestamp
importance	low / normal / high

The visualization should consume state/events, not invent authoritative state. If the backend says an agent is asleep, the visual client should display that state.

12. Event Protocol

```
Use a simple JSON event contract initially. Example:
```

```
{
"event": "agent.status",
"timestamp": "2026-09-05T20:00:00",
"agent_id": "research_01",
"role": "Research Agent",
"state": "working",
"task_id": "task_102",
"progress": 0.45,
"message": "Collecting sources"
}
```

Suggested events: agent.created, agent.activated, agent.status, agent.completed, agent.sleeping, task.created, task.progress, task.completed, memory.saved, lesson.candidate, lesson.verified, system.warning and system.error.

13. Mock Mode — Build This First

Before networking, create a local mock event generator. It should simulate 3–5 roles changing state. This allows the entire visual client to be tested without Project 2.

```
Example sequence:
```

```
Research Agent: idle → working → waiting → completed → sleeping
Software Engineer: sleeping → working → testing → completed
Teacher: idle → teaching → assessment → waiting
```

> This separation is critical: Project 1 must be testable before Project 2 exists.

14. Development Steps

1. Create a Godot project.
2. Create the main scene and camera.
3. Create a simple grid/environment.
4. Create one reusable agent visual.
5. Add role labels and state indicators.
6. Add five mock agents.
7. Add task visualization.
8. Add event timeline.
9. Add selected-agent information panel.

10 Implement mock event playback.

11 Add JSON event parsing.

12 Add network transport only after mock mode is stable.

13 Add reconnect/error behavior.

14 Optimize geometry, draw calls and update frequency.

15 Create Windows and Web exports.

15. Networking Options

For the first version, HTTP polling is easier to understand. Later, WebSocket can provide real-time events. The backend can expose an event endpoint or a stream; Project 1 only renders the information it receives.

Method	When to use	Complexity
Mock JSON	Learning/testing	Very low
HTTP polling	Simple integration	Low
WebSocket	Real-time event stream	Medium
Local file replay	Demo/debugging	Low

16. Performance Rules

- Keep geometry simple.
- Use low-resolution textures or no textures.
- Use a small number of materials.
- Avoid unnecessary real-time shadows.
- Do not update every visual element every frame if state changes slowly.
- Use object reuse rather than constantly creating/destroying nodes.
- Limit event history shown on screen.
- Test at 1280×720 and 1920×1080.
- Measure RAM/GPU usage before adding visual complexity.

17. Testing

- Start with no backend: mock mode works.
- Malformed JSON does not crash the application.
- Unknown agent IDs are handled.
- Unknown event types are ignored/logged safely.
- Network disconnect displays a clear status.
- Reconnect does not duplicate events.
- Large event bursts do not freeze the UI.
- Sleeping agents become visually inactive.
- Windows export launches on a clean machine.
- Web export loads through a static host.

18. Free Deployment

Primary desktop deployment: export a Windows executable and publish the ZIP through GitHub Releases. No server is required.

Optional web deployment: export the Godot Web build and publish the static files through GitHub Pages. GitHub Pages is a static hosting service; it cannot run your AI backend. This is therefore appropriate for Project 1 only when the visualization can run with mock data or call a separately hosted/public backend.

```
Godot supports command-line release export using an export preset. Example:
godot --path godot --export-release "Windows Desktop" builds/AIVisualization.exe
```

19. Web Export Caveats

- Web builds are constrained by browser security and available browser APIs.
- Use a simple static deployment first.
- Keep backend communication CORS-safe when a web client calls an API.
- Do not put private API keys inside the web build.
- Prefer mock/demo mode for public visualization if the backend is private.

20. Project 1 Completion Checklist

- Godot project runs on the target laptop.
- Stylized environment works.
- Reusable role/agent visual works.
- Tasks and states are visible.
- Mock event stream works.
- JSON event parsing works.
- Network disconnect/reconnect is handled.
- Windows build works.
- Optional Web build works.
- Project 1 remains independent from Project 2.

21. Official/Reference Resources

Godot 4.7.2 stable release: https://godotengine.org/download/archive/4.7.2-stable/

Godot documentation: https://docs.godotengine.org/en/stable/

Godot export documentation: https://docs.godotengine.org/en/stable/tutorials/export/exporting_projects.html

Blender: https://www.blender.org/download/

Blender license: https://www.blender.org/about/license/

GitHub Pages: https://docs.github.com/en/pages

Git: https://git-scm.com/

VS Code: https://code.visualstudio.com/

> These are selected because they are free/open-source or provide the required free hosting/distribution path. Free-tier limits can change, so verify the current limits before deployment.